Chapter 2 — SELECT Basics: Reading, Filtering, Sorting and Paging

Reading data is most of what anyone does with SQL. This chapter teaches the single-table `SELECT` statement completely: choosing columns, computing new ones, filtering rows with `WHERE`, the operators you can use there, the peculiar but essential rules of `NULL`, removing duplicates with `DISTINCT`, sorting with `ORDER BY`, and returning one page of results with `LIMIT`/`OFFSET` or the standard `FETCH`. Joins, grouping and subqueries build on exactly these pieces in chapters 4–6.

What you'll learn

#	Topic	You will be able to…
2.1	`SELECT` and aliases	pick columns, compute expressions, rename outputs
2.2	`WHERE` and logical operators	filter rows and combine conditions without precedence bugs
2.3	Operators	use arithmetic, comparison, logical and string operators correctly (incl. integer division)
2.4	`BETWEEN`, `IN`, `LIKE`/`ILIKE`, regex	express ranges, lists and text patterns
2.5	`NULL` and three-valued logic	test for missing values and avoid the classic `NOT IN` trap
2.6	`DISTINCT` / `DISTINCT ON`	remove duplicates and keep one row per group
2.7	`ORDER BY`	sort by several keys, control where NULLs go, custom orders
2.8	`LIMIT`/`OFFSET`/`FETCH`	page through results, handle ties, prefer keyset paging
2.9	Logical processing order	know why an alias works in `ORDER BY` but not in `WHERE`

Setup. All examples use the `company` sample database from chapter 1, section 1.11. Files are in `code/`; each starts with `\pset null '(null)'` so missing values print as `(null)` instead of blanks. To follow along: `bash tools_pg.sh psql -d company`, then type `\pset null '(null)'`.

2.1 SELECT: choosing what comes back

What it is

`SELECT` reads rows from a table and returns a new, temporary table called the **result set**. Think of the table as a spreadsheet: `SELECT` chooses which columns you see, and (with `WHERE`, next section) which rows.

```
SELECT column1, column2, ...     -- what to show
FROM   table_name;               -- where to read it
```

How it works

1. `FROM` names the table to read.
2. The `SELECT` list says which columns (or computed values) to return, in which order.
3. `*` is a wildcard meaning "every column, in table order".

All columns:

```sql
-- code/01_select_columns.sql
SELECT * FROM products;
```

Output:

```
 product_id |            name             |  category   | unit_price | discontinued
------------+-----------------------------+-------------+------------+--------------
          1 | Laptop Pro 14               | Electronics |    1299.00 | f
          2 | Wireless Mouse              | Electronics |      24.99 | f
          3 | USB-C Hub                   | Accessories |      49.50 | f
          4 | Mechanical Keyboard         | Accessories |      89.00 | f
          5 | Standing Desk               | Furniture   |     499.00 | f
          6 | Ergonomic Chair             | Furniture   |     349.99 | f
          7 | Noise-Cancelling Headphones | Electronics |     199.95 | f
          8 | Desk Lamp                   | Furniture   |      35.00 | t
          9 | Gift Card                   | (null)      |      50.00 | f
(9 rows)
```

(`f` / `t` are how psql prints boolean false/true.)

Chosen columns — list them after `SELECT`, in the order you want:

```sql
SELECT first_name, last_name, job_title FROM employees;
```

Output (first rows):

```
 first_name | last_name |      job_title
------------+-----------+---------------------
 Asha       | Rao       | CEO
 Vikram     | Menon     | Engineering Manager
 Neha       | Gupta     | Senior Engineer
 Rahul      | Verma     | Engineer
 ...
(15 rows)
```

Expressions and column aliases

The `SELECT` list can hold any **expression** — arithmetic, function calls, constants — not just column names. An **alias** (`AS new_name`) names the output column. Use double quotes if the alias has spaces or capitals you want to keep.

```sql
SELECT name,
       unit_price,
       unit_price * 1.18        AS price_with_gst,
       'per unit'               AS note,
       unit_price * 1.18 AS "Price incl. GST"
FROM products
WHERE product_id <= 3;
```

Output:

```
      name      | unit_price | price_with_gst |   note   | Price incl. GST
----------------+------------+----------------+----------+-----------------
 Laptop Pro 14  |    1299.00 |      1532.8200 | per unit |       1532.8200
 Wireless Mouse |      24.99 |        29.4882 | per unit |         29.4882
 USB-C Hub      |      49.50 |        58.4100 | per unit |         58.4100
(3 rows)
```

The table is not changed — computed columns exist only in the result. (Rounding to 2 decimals is `round(x, 2)`, chapter 3.)

Table aliases give a table a short name, which you'll need constantly once queries involve several tables. The `AS` keyword is optional for both kinds of alias, but writing it for column aliases avoids confusing bugs (see *Common mistakes*).

```sql
SELECT e.first_name, e.salary / 12 monthly
FROM employees AS e
WHERE e.emp_id IN (1, 2);
```

Output:

```
 first_name |      monthly
------------+--------------------
 Asha       | 20833.333333333333
 Vikram     | 15416.666666666667
(2 rows)
```

SELECT without FROM — PostgreSQL (and MySQL, SQLite, SQL Server) let you evaluate expressions directly, which makes psql a handy calculator. Oracle required `FROM dual` until version 23ai made `FROM` optional.

```sql
SELECT 2 + 3 AS five, now()::date >= DATE '2025-01-01' AS after_2025;
```

Output:

```
 five | after_2025
------+------------
    5 | t
(1 row)
```

Common mistakes

- **`SELECT *` in application code.** It returns columns you don't need (more I/O and network), and code breaks or silently changes when someone adds a column. Name the columns. `SELECT *` is fine for exploring in psql.
- **A missing comma turns a column into an alias.** `SELECT first_name last_name FROM employees` returns *one* column named `last_name` containing first names. Writing `AS` for every alias makes this mistake visible.
- **Quoting aliases with single quotes.** `'total'` is a string literal; aliases use double quotes: `"Total"`.

Self-check

1. *What does `SELECT *` return for a table with 5 columns?* → All 5 columns in table-definition order.
2. *Does `SELECT unit_price * 2 FROM products` change prices?* → No; it only computes a value in the result set.

3. *How do you name an output column `Net Price` exactly?* → `AS "Net Price"`.

TL;DR

`SELECT cols FROM table`. Use `*` only to explore. Any expression can be a column; `AS alias` names it; table aliases shorten references.

2.2 WHERE: filtering rows

What it is

`WHERE condition` keeps only the rows for which the condition is **true**. It works the same way in `SELECT`, `UPDATE` and `DELETE` statements (chapter 9), so a filter you test with `SELECT` can be reused safely in a `DELETE`.

```
SELECT column1, column2
FROM   table_name
WHERE  condition;
```

How it works

For each row, the database evaluates the condition. The result is `TRUE`, `FALSE` or — when `NULL` is involved — `UNKNOWN` (section 2.5). Only `TRUE` rows pass.

```
-- code/02_where.sql
SELECT name, category, unit_price
FROM products
WHERE category = 'Electronics';
```

Output:

```
            name             |  category   | unit_price
-----------------------------+-------------+------------
 Laptop Pro 14               | Electronics |    1299.00
 Wireless Mouse              | Electronics |      24.99
 Noise-Cancelling Headphones | Electronics |     199.95
(3 rows)
```

Comparisons work on numbers, text and dates alike. Date literals are written as strings in ISO format `'YYYY-MM-DD'`, which PostgreSQL converts to the column's type:

```
SELECT first_name, hire_date FROM employees WHERE hire_date >= '2023-01-01' ORDER BY hire_date;
```

Output:

```
 first_name | hire_date
------------+------------
 Li         | 2023-02-14
 Arjun      | 2023-11-20
 Sara       | 2024-04-01
 Rohan      | 2025-06-02
(4 rows)
```

Combining conditions: AND, OR, NOT

Operator	True when	Example
`AND`	both sides are true	`dept_id = 2 AND salary > 70000`
`OR`	at least one side is true	`city = 'Delhi' OR city = 'Mumbai'`
`NOT`	the condition is false	`NOT discontinued`

Precedence: `NOT` binds tightest, then `AND`, then `OR` — just as `*` binds before `+`. So `a OR b AND c` means `a OR (b AND c)`. This silently changes results:

```
-- Intended: Engineering or Sales staff earning over 100k
SELECT first_name, dept_id, salary
FROM employees
WHERE dept_id = 1 OR dept_id = 2 AND salary > 100000
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Rahul      |       1 |  98000.00
 Meera      |       1 |  98000.00
 Daniel     |       2 | 150000.00
(5 rows)
```

Rahul and Meera earn less than 100k but appear, because the query really says "*everyone* in dept 1, or dept-2 staff over 100k". Parentheses fix it:

```
SELECT first_name, dept_id, salary
FROM employees
WHERE (dept_id = 1 OR dept_id = 2) AND salary > 100000
ORDER BY emp_id;
```

Output:

```
 first_name | dept_id |  salary
------------+---------+-----------
 Vikram     |       1 | 185000.00
 Neha       |       1 | 142000.00
 Daniel     |       2 | 150000.00
(3 rows)
```

`NOT` negates the condition after it:

```
SELECT name FROM products WHERE NOT discontinued AND NOT unit_price < 100 ORDER BY name;
```

Output:

```
            name
----------------------------
 Ergonomic Chair
 Laptop Pro 14
 Noise-Cancelling Headphones
 Standing Desk
(4 rows)
```

Case sensitivity

In PostgreSQL, text comparison with `=` is **case-sensitive**: `'Mumbai' = 'mumbai'` is false. Our customers table has one `Mumbai` and one `mumbai`, and `WHERE city = 'Mumbai'` finds only one:

```
 exact_mumbai
--------------
            1
```

MySQL's default collations are case-*insensitive*, so the same query there returns 2 — a real portability trap. In PostgreSQL, compare `lower(city) = 'mumbai'` or use `ILIKE` (2.4).

Common mistakes

- Mixing `AND` and `OR` without parentheses. Rule of thumb: whenever both appear, add parentheses.
- Comparing with `= NULL` — never true; use `IS NULL` (2.5).
- Writing `"Mumbai"` (double quotes) for a string: PostgreSQL reads it as a column name and errors with *column "Mumbai" does not exist*.

Self-check

1. *How is `a AND b OR c` evaluated?* → `(a AND b) OR c`.
2. *Does `WHERE` also apply to `DELETE`?* → Yes; `DELETE FROM t WHERE …` removes only matching rows (and without `WHERE`, all rows).
3. *Why does `WHERE city = 'Mumbai'` miss `'mumbai'` in PostgreSQL?* → String equality is case-sensitive.

TL;DR

`WHERE` keeps rows whose condition is TRUE. `NOT` > `AND` > `OR` in precedence — parenthesise mixed conditions. Text equality is case-sensitive in PostgreSQL.

2.3 Operators

What it is

An **operator** is a symbol or keyword that combines one or more values (its **operands**) into a new value: `+`, `=`, `AND`, `||`.

Arithmetic operators

Operator	Meaning	Example	Result
`+`	add	`5 + 3`	8
`-`	subtract	`10 - 4`	6
`*`	multiply	`3 * 7`	21
`/`	divide	`20 / 5`	4
`%`	remainder (modulo)	`17 % 3`	2
`^`	power (PostgreSQL)	`2 ^ 10`	1024
`|/`, `@`	square root, absolute value (PostgreSQL)	`|/ 25`, `@ -5`	5, 5

```sql
-- code/08_operators.sql
SELECT 5 + 3 AS add, 10 - 4 AS sub, 3 * 7 AS mul, 20 / 5 AS div, 17 % 3 AS modulo;
SELECT 7 / 2 AS int_div, 7 / 2.0 AS num_div, 7::numeric / 2 AS cast_div, -7 / 2 AS neg_int_div;
SELECT 2 ^ 10 AS power, |/ 25 AS sqrt, @ -5 AS abs;
SELECT 2 + 3 * 4 AS no_parens, (2 + 3) * 4 AS parens;
```

Output:

```
 add | sub | mul | div | modulo
-----+-----+-----+-----+--------
   8 |   6 |  21 |   4 |      2
(1 row)

 int_div |      num_div       |      cast_div      | neg_int_div
---------+--------------------+--------------------+-------------
       3 | 3.5000000000000000 | 3.5000000000000000 |          -3
(1 row)

 power | sqrt | abs
-------+------+-----
  1024 |    5 |   5
(1 row)

 no_parens | parens
-----------+--------
        14 |     20
(1 row)
```

Integer division truncates toward zero in PostgreSQL, SQL Server and SQLite: `7 / 2` is `3`, and `-7 / 2` is `-3`. If either side is a decimal (`2.0`) or cast to `numeric`, you get the fraction. MySQL is the odd one out: `/` always returns a decimal (`7 / 2` = `3.5000`), and `DIV` does integer division. This is one of the most common "my average is wrong" bugs when moving between databases.

Operators work on columns row by row — for example the value of each order line:

```sql
SELECT order_id, product_id, quantity, unit_price, discount,
       quantity * unit_price * (1 - discount) AS line_total
FROM order_items WHERE order_id IN (102, 104) ORDER BY order_id, product_id;
```

Output:

```
 order_id | product_id | quantity | unit_price | discount | line_total
----------+------------+----------+------------+----------+------------
      102 |          7 |        2 |     199.95 |     0.10 |   359.9100
      104 |          5 |        1 |     479.00 |     0.00 |   479.0000
      104 |          6 |        1 |     349.99 |     0.05 |   332.4905
(3 rows)
```

Comparison operators

Operator	Meaning	Example
`=`	equal	`unit_price = 49.50`
`<>` or `!=`	not equal (`<>` is standard)	`status <> 'cancelled'`
`<`, `>`, `<=`, `>=`	ordering	`salary >= 100000`
`BETWEEN a AND b`	inclusive range	2.4
`IN (…)`	membership in a list	2.4
`LIKE`, `ILIKE`, `~`	pattern match	2.4
`IS [NOT] NULL`, `IS [NOT] DISTINCT FROM`	NULL-aware tests	2.5

Logical operators

`AND`, `OR`, `NOT` — see 2.2 for precedence and 2.5 for how they behave with `NULL`.

String concatenation

`||` joins strings in PostgreSQL, SQLite and Oracle:

```sql
SELECT first_name || ' ' || last_name AS full_name FROM employees WHERE emp_id <= 3;
```

Output:

```
  full_name
--------------
 Asha Rao
 Vikram Menon
 Neha Gupta
(3 rows)
```

In MySQL `||` means logical OR by default; use `CONCAT()` there. SQL Server uses `+` or `CONCAT()`. Chapter 3 covers `concat()` and how each treats `NULL`.

Compound assignment operators

Some SQL dialects have shorthand assignment operators, e.g. SQL Server's T-SQL: `SET @x += 3` (add and assign), also `-=`, `*=`, `/=`, `%=`. They apply to **procedural variables** (and T-SQL's `UPDATE … SET col += 1`). They are **not** part of standard SQL and PostgreSQL does not support them in SQL statements — write `SET col = col + 1` in an `UPDATE`, or `x := x + 3` inside a PL/pgSQL function (chapter 14).

Dialect	Add-and-assign
PostgreSQL	`UPDATE t SET n = n + 3`; PL/pgSQL `x := x + 3`
SQL Server	`SET @x += 3`; `UPDATE t SET n += 3`
MySQL	`SET @x = @x + 3` (no `+=`)

Operator precedence (highest first, simplified)

Level	Operators
1	`::` (cast)
2	unary `-` / `+`
3	`^`
4	`*`, `/`, `%`
5	binary `+`, `-`
6	other operators such as `||`
7	`BETWEEN`, `IN`, `LIKE`, `ILIKE`
8	`<`, `>`, `=`, `<=`, `>=`, `<>`
9	`IS NULL`, `IS DISTINCT FROM` …
10	`NOT`
11	`AND`
12	`OR`

Because unary minus binds tighter than `^`, `SELECT -2 ^ 2` returns `4` in PostgreSQL, not `-4`. When in doubt, add parentheses — they cost nothing and document intent.

Self-check

1. *`SELECT 9 / 4` in PostgreSQL?* → `2` (integer division). `9 / 4.0` → `2.25`.
2. *How do you write "not equal" in standard SQL?* → `<>` (PostgreSQL also accepts `!=`).
3. *Why might `first_name || ' ' || last_name` return nothing useful in MySQL?* → `||` is logical OR there by default.

TL;DR

`+ - * / %` (and PostgreSQL's `^`), with integer division truncating. `<>` for not-equal. `||` concatenates in PostgreSQL. Compound `+=` is a T-SQL/procedural feature, not standard SQL.

2.4 Ranges, lists and patterns: BETWEEN, IN, LIKE/ILIKE, regex

BETWEEN — inclusive ranges

`x BETWEEN a AND b` means `x >= a AND x <= b`. **Both ends are included.**

```sql
-- code/03_between_in_like.sql
SELECT name, unit_price FROM products WHERE unit_price BETWEEN 49.50 AND 199.95 ORDER BY unit_price;
```

Output:

```
            name             | unit_price
-----------------------------+------------
 USB-C Hub                   |      49.50
 Gift Card                   |      50.00
 Mechanical Keyboard         |      89.00
 Noise-Cancelling Headphones |     199.95
(4 rows)
```

`NOT BETWEEN a AND b` is the complement. `a` must be the smaller value; `BETWEEN 200 AND 50` matches nothing (PostgreSQL's `BETWEEN SYMMETRIC` accepts either order).

The timestamp trap. `placed_at` is a timestamp (date *and* time). The string `'2025-01-31'` becomes midnight at the *start* of 31 January, so an order at 23:55 that day is excluded:

```
SELECT order_id, placed_at FROM orders
WHERE placed_at BETWEEN '2025-01-01' AND '2025-01-31'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
(2 rows)
```

Order 103 (31 Jan, 23:55) is missing. The robust pattern for any time range is **half-open**: `>=` the start and `<` the next period's start.

```
SELECT order_id, placed_at FROM orders
WHERE placed_at >= '2025-01-01' AND placed_at < '2025-02-01'
ORDER BY order_id;
```

Output:

```
 order_id |      placed_at
----------+---------------------
      101 | 2025-01-03 10:15:00
      102 | 2025-01-17 22:40:00
      103 | 2025-01-31 23:55:00
(3 rows)
```

IN — membership in a list

`x IN (a, b, c)` is shorthand for `x = a OR x = b OR x = c`; `NOT IN` is its negation.

```
SELECT order_id, status FROM orders WHERE status IN ('pending', 'cancelled') ORDER BY order_id;
SELECT first_name, job_title FROM employees WHERE job_title NOT IN ('Engineer', 'Sales Rep', 'Intern')
ORDER BY emp_id;
```

Output:

```
 order_id |  status
----------+-----------
      105 | cancelled
      111 | cancelled
      113 | pending
      114 | pending
(4 rows)

 first_name |      job_title
------------+---------------------
 Asha       | CEO
 Vikram     | Engineering Manager
 Neha       | Senior Engineer
 Daniel     | Sales Manager
 Lucas      | Marketing Lead
 Ananya     | Content Writer
 Kiran      | Finance Manager
 Omar       | Accountant
 Li         | Support Engineer
(9 rows)
```

`IN` can also take a subquery (`WHERE customer_id IN (SELECT …)`) — chapter 6. **Warning:** `NOT IN` with a `NULL` in the list returns no rows at all; see 2.5.

LIKE and ILIKE — simple patterns

`LIKE` matches text against a pattern with two wildcards:

Wildcard	Matches	Example	Matches…
`%`	any sequence of zero or more characters	`'%Desk%'`	"Standing Desk", "Desk Lamp"
`_`	exactly one character	`'_a%'`	"Daniel", "Rahul", "Sara" (second letter *a*)

```sql
SELECT name FROM products WHERE name LIKE '%Desk%' ORDER BY name;
SELECT first_name FROM employees WHERE first_name LIKE '_a%' ORDER BY first_name;
```

Output:

```
     name
---------------
 Desk Lamp
 Standing Desk
(2 rows)

 first_name
------------
 Daniel
 Rahul
 Sara
(3 rows)
```

`LIKE` is **case-sensitive** in PostgreSQL. `ILIKE` (PostgreSQL-only) ignores case:

```sql
SELECT email FROM employees WHERE email LIKE 'neha%';
SELECT email FROM employees WHERE email ILIKE 'neha%';
```

Output:

```
 email
-------
(0 rows)

           email
----------------------------
 Neha.Gupta@Company.example
(1 row)
```

Portable alternative: `lower(email) LIKE 'neha%'`.

Matching a literal `%` or `_` — escape it. PostgreSQL's default escape character is backslash; `ESCAPE` chooses another:

```
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10\%%' ORDER BY label;
SELECT label FROM (VALUES ('10% off'), ('100 off'), ('10_off')) AS t(label)
WHERE label LIKE '10!_%' ESCAPE '!' ORDER BY label;
```

Output:

```
  label
---------
 10% off
(1 row)

 label
--------
 10_off
(1 row)
```

(`VALUES (…), (…) AS t(label)` builds a tiny inline table — handy for experiments.)

Regular expressions (PostgreSQL)

For richer patterns PostgreSQL offers POSIX regular expressions: `~` (case-sensitive match), `~*` (case-insensitive), `!~` and `!~*` (no match). Names made of a capitalised word, a space, then another capital:

```
SELECT name FROM products WHERE name ~ '^[A-Z][a-z]+ [A-Z]' ORDER BY name;
```

Output:

```
        name
--------------------
 Desk Lamp
 Ergonomic Chair
 Gift Card
 Laptop Pro 14
 Mechanical Keyboard
 Standing Desk
 Wireless Mouse
(7 rows)
```

`USB-C Hub` fails (`S` isn't lower case) and `Noise-Cancelling…` fails (hyphen, not space). The standard's `SIMILAR TO` also exists but is rarely used.

Performance note

`LIKE 'abc%'` (a fixed prefix) can use a B-tree index (in PostgreSQL the index needs the `text_pattern_ops` operator class unless the database uses the `C` collation); `LIKE '%abc'` with a leading wildcard cannot, and scans every row. Chapter 12 shows trigram (`pg_trgm`) indexes for "contains" searches.

Dialect notes

Feature	PostgreSQL	MySQL	SQLite	SQL Server
`LIKE` case-sensitivity	sensitive	depends on collation (default insensitive)	insensitive for ASCII by default	depends on collation (default insensitive)
Case-insensitive LIKE	`ILIKE`	`LIKE` (default)	`LIKE`	`LIKE` (default)
Regex	`~`, `~*`	`REGEXP` / `RLIKE`	`REGEXP` only when a `regexp()` function is loaded (the `sqlite3` shell includes one)	none built in (SQL Server 2025 adds `REGEXP_LIKE`)

Common mistakes

- `BETWEEN` on timestamps with date-only upper bounds (use half-open ranges).
- Forgetting that `_` is a wildcard: `LIKE 'user_%'` also matches `userA…`.
- Expecting `LIKE` to be case-insensitive in PostgreSQL.

Self-check

1. *Is `BETWEEN 10 AND 20` inclusive of 20?* → Yes, both ends.
2. *Pattern for "exactly 4 characters ending in `n`"?* → `LIKE '___n'`.
3. *How do you match names containing `%`?* → `LIKE '%\%%'` (or `ESCAPE`).

TL;DR

`BETWEEN` is inclusive (use `>=` / `<` for timestamps), `IN` is shorthand for ORs, `LIKE` uses `%` and `_` and is case-sensitive in PostgreSQL (`ILIKE` isn't), regex via `~`.

2.5 NULL and three-valued logic

What it is

`NULL` is a **marker for a missing value** — unknown ("we don't know Meera's email") or not applicable ("interns earn no commission"). It is not zero, not an empty string, and not a value at all, so it cannot be *equal* to anything — not even another `NULL`.

Why it matters

Real data is full of gaps, and `NULL` changes how every comparison, filter and calculation behaves. Most "missing rows" bugs in SQL come from it.

How it works — three-valued logic

Any comparison involving `NULL` yields the third truth value **UNKNOWN** (displayed as `NULL`). Arithmetic and concatenation with `NULL` yield `NULL`.

```sql
-- code/04_nulls.sql
SELECT NULL = NULL AS null_eq_null, NULL <> 1 AS null_ne_1, NULL IS NULL AS is_null,
       1 + NULL AS plus, 'a' || NULL AS concat;
```

Output:

```
 null_eq_null | null_ne_1 | is_null |  plus  | concat
--------------+-----------+---------+--------+--------
 (null)       | (null)    | t       | (null) | (null)
(1 row)
```

`AND`, `OR` and `NOT` follow these truth tables (generated by the database itself):

```sql
SELECT a, b, a AND b AS "a AND b", a OR b AS "a OR b", NOT a AS "NOT a"
FROM (VALUES (true), (false), (NULL::boolean)) AS x(a)
CROSS JOIN (VALUES (true), (false), (NULL::boolean)) AS y(b);
```

Output:

```
   a    |   b    | a AND b | a OR b | NOT a
--------+--------+---------+--------+--------
 t      | t      | t       | t      | f
 t      | f      | f       | t      | f
 t      | (null) | (null)  | t      | f
 f      | t      | f       | t      | t
 f      | f      | f       | f      | t
 f      | (null) | f       | (null) | t
 (null) | t      | (null)  | t      | (null)
 (null) | f      | f       | (null) | (null)
 (null) | (null) | (null)  | (null) | (null)
(9 rows)
```

Read it as "UNKNOWN might be either true or false": `FALSE AND UNKNOWN` is `FALSE` whatever the unknown is; `TRUE OR UNKNOWN` is `TRUE`; anything else involving UNKNOWN stays UNKNOWN. And `NOT UNKNOWN` is still UNKNOWN.

`WHERE` **keeps only TRUE** — both FALSE and UNKNOWN rows are dropped.

Testing for NULL: IS NULL / IS NOT NULL

```sql
SELECT count(*) AS eq_null  FROM employees WHERE email = NULL;
SELECT count(*) AS is_null  FROM employees WHERE email IS NULL;
SELECT first_name, last_name FROM employees WHERE email IS NULL ORDER BY emp_id;
```

Output:

```
 eq_null
---------
       0
(1 row)

 is_null
---------
       2
(1 row)

 first_name | last_name
------------+-----------
 Meera      | Iyer
 Ananya     | Das
(2 rows)
```

`= NULL` is always UNKNOWN, so it never matches. (SQL Server has a legacy `ANSI_NULLS OFF` mode where `= NULL` works; don't rely on it.)

The "missing rows" trap

Sales reps have commission `0.10`, `0.10` and `NULL`. Asking for `= 0.10` and then `<> 0.10` should cover everyone — but Sara (NULL) appears in neither:

```
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct =
0.10 ORDER BY emp_id;
SELECT first_name, commission_pct FROM employees WHERE job_title = 'Sales Rep' AND commission_pct <>
0.10 ORDER BY emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Priya      |           0.10
 Arjun      |           0.10
(2 rows)

 first_name | commission_pct
------------+----------------
(0 rows)
```

`IS DISTINCT FROM` is the NULL-safe "not equal": it treats two NULLs as equal and NULL vs a value as different, and always returns TRUE or FALSE.

```
SELECT first_name, commission_pct FROM employees
WHERE job_title = 'Sales Rep' AND commission_pct IS DISTINCT FROM 0.10 ORDER BY emp_id;
```

Output:

```
 first_name | commission_pct
------------+----------------
 Sara       |         (null)
(1 row)
```

(MySQL spells NULL-safe equality `<=>`; SQL Server supports `IS [NOT] DISTINCT FROM` from version 2022.)

The NOT IN trap

`x NOT IN (1, 2, NULL)` expands to `x <> 1 AND x <> 2 AND x <> NULL`. The last part is always UNKNOWN, so the whole condition can never be TRUE:

```
SELECT count(*) AS not_in_with_null FROM departments WHERE dept_id NOT IN (1, 2, NULL);
SELECT count(*) AS not_in_no_null   FROM departments WHERE dept_id NOT IN (1, 2);
```

Output:

```
 not_in_with_null
------------------
                0
(1 row)

 not_in_no_null
----------------
              4
(1 row)
```

This bites hardest with subqueries — `WHERE id NOT IN (SELECT manager_id FROM employees)` returns nothing because the CEO's `manager_id` is NULL. Use `NOT EXISTS` instead (chapter 6).

NULL is not zero or ''

```
SELECT count(*) AS rows, count(commission_pct) AS non_null_commissions FROM employees;
SELECT '' IS NULL AS empty_string_is_null;
```

Output:

```
 rows | non_null_commissions
------+----------------------
   15 |                    3
(1 row)

 empty_string_is_null
----------------------
 f
(1 row)
```

`count(*)` counts rows; `count(column)` counts non-NULL values (chapter 4). And in PostgreSQL an empty string is a real value, not NULL. (**Oracle** is the exception: it stores `''` as NULL.)

Allowing or forbidding NULL in a column

Columns accept NULL **unless** declared `NOT NULL`. Writing `NULL` in a column definition just states the default explicitly.

```
CREATE TABLE students (
    id    integer PRIMARY KEY,
    email text NULL,          -- explicit "NULL allowed" (the default anyway)
    name  text NOT NULL       -- a value is required
);
INSERT INTO students VALUES (1, NULL, 'Ravi');
INSERT INTO students VALUES (2, 'kim@school.example', NULL);
SELECT * FROM students WHERE email IS NULL;
```

Output:

```
CREATE TABLE
INSERT 0 1
ERROR:  null value in column "name" of relation "students" violates not-null constraint
DETAIL:  Failing row contains (2, kim@school.example, null).
 id | email  | name
----+--------+------
  1 | (null) | Ravi
(1 row)
```

Replacing NULLs in output (`COALESCE`, `NULLIF`) is in chapter 3; NULLs in sorting are in 2.7, in `DISTINCT` in 2.6, in aggregates in chapter 4, in joins in chapter 5.

In the interview

30-second answer: "NULL means unknown, so comparisons with it give UNKNOWN, and WHERE only keeps TRUE rows. Use IS NULL / IS NOT NULL, or IS DISTINCT FROM for NULL-safe comparison. Watch NOT IN with a NULL in the list or subquery — it returns nothing — so prefer NOT EXISTS. Aggregates like COUNT(col) and SUM skip NULLs."

- *What does* `NULL = NULL` *return?* → NULL (UNKNOWN), not TRUE.
- *Why can* `WHERE x <> 5` *miss rows?* → Rows where `x` is NULL give UNKNOWN and are filtered out.
- *Is* `''` *NULL?* → Not in PostgreSQL, MySQL, SQL Server or SQLite; yes in Oracle.

Self-check

1. `TRUE OR NULL`*?* → TRUE.
2. `FALSE AND NULL`*?* → FALSE.
3. `NOT NULL` *(as a boolean expression)?* → NULL.
4. *How many rows does* `WHERE email = NULL` *return?* → Zero, always.

TL;DR

NULL = unknown. Comparisons with NULL are UNKNOWN; `WHERE` keeps only TRUE. Test with `IS [NOT] NULL`; compare NULL-safely with `IS [NOT] DISTINCT FROM`; never put a nullable list inside `NOT IN`.

2.6 DISTINCT: removing duplicate rows

What it is

`SELECT DISTINCT` removes duplicate rows from the **result** (the table itself is untouched). It answers questions like "which categories exist?" or "which cities do our customers live in?".

```
SELECT DISTINCT column1, column2
FROM table_name
WHERE condition;
```

How it works

```
-- code/05_distinct.sql
SELECT category FROM products ORDER BY category;
SELECT DISTINCT category FROM products ORDER BY category;
```

Output:

```
  category
-------------
 Accessories
 Accessories
 Electronics
 Electronics
 Electronics
 Furniture
 Furniture
 Furniture
 (null)
(9 rows)

  category
-------------
 Accessories
 Electronics
 Furniture
 (null)
(4 rows)
```

Two rules to notice:

1. **NULLs count as duplicates of each other** for `DISTINCT` (unlike `=`) — all NULL categories collapse into one row.
2. `DISTINCT` **applies to the whole selected row**, not to the first column. With two columns, you get distinct *pairs*:

```
SELECT DISTINCT country, city FROM customers ORDER BY country, city;
```

Output:

```
 country |   city
---------+-----------
 India   | Bengaluru
 India   | Delhi
 India   | Mumbai
 India   | mumbai
 India   | (null)
 UK      | London
 USA     | New York
(7 rows)
```

`Mumbai` and `mumbai` are different strings, so both survive — `DISTINCT` removes only **exact** duplicates. Dirty data needs cleaning first. Our customer Anita Shah was entered twice with differently formatted emails:

```
SELECT DISTINCT first_name, last_name, email FROM customers WHERE last_name = 'Shah';
SELECT DISTINCT first_name, last_name, lower(trim(email)) AS email FROM customers WHERE last_name =
'Shah';
```

Output:

```
 first_name | last_name |           email
------------+-----------+---------------------------
 Anita      | Shah      |   Anita.Shah@Mail.example
 Anita      | Shah      | anita.shah@mail.example
(2 rows)

 first_name | last_name |          email
------------+-----------+-------------------------
 Anita      | Shah      | anita.shah@mail.example
(1 row)
```

Counting distinct values uses `count(DISTINCT expr)`, which ignores NULLs (chapter 4 covers aggregates):

```
SELECT count(*) AS rows, count(city) AS with_city, count(DISTINCT city) AS distinct_cities,
       count(DISTINCT lower(city)) AS distinct_cities_ci
FROM customers;
```

Output:

```
 rows | with_city | distinct_cities | distinct_cities_ci
------+-----------+-----------------+--------------------
   10 |         9 |               6 |                  5
(1 row)
```

DISTINCT ON (PostgreSQL)

`DISTINCT ON (expr)` keeps the **first row of each group** of equal `expr` values, where "first" is decided by `ORDER BY` (which must start with the same expressions). A classic use is "the top earner in each department":

```
SELECT DISTINCT ON (dept_id) dept_id, first_name, salary
FROM employees
WHERE dept_id IS NOT NULL
ORDER BY dept_id, salary DESC, emp_id;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       2 | Daniel     | 150000.00
       3 | Lucas      | 120000.00
       4 | Kiran      | 135000.00
       5 | Li         |  64000.00
(5 rows)
```

Other databases solve "first row per group" with window functions (`ROW_NUMBER()`, chapter 8).

Trade-offs and when not to use it

- `DISTINCT` needs a sort or hash of the whole result — cheap here, expensive on millions of rows.
- **Don't use `DISTINCT` to hide a bad join.** If a join produces duplicates you didn't expect, the join condition is probably wrong (chapter 5 calls this *fan-out*); `DISTINCT` masks the symptom and can hide real duplicates too.
- If you need counts per value, you want `GROUP BY` (chapter 4), not `DISTINCT`.

Self-check

1. *Does `SELECT DISTINCT a, b` make `a` unique?* → No; it makes each `(a, b)` pair unique.
2. *Do two NULLs count as duplicates under DISTINCT?* → Yes.
3. *Why did `count(DISTINCT city)` give 6 but the case-insensitive version 5?* → `Mumbai` and `mumbai` differ as strings but match after `lower()`.

TL;DR

`DISTINCT` removes exact duplicate result rows (NULLs grouped together); it applies to the whole select list. Clean data (`lower`, `trim`) before de-duplicating; use PostgreSQL's `DISTINCT ON` for "first row per group".

2.7 ORDER BY: sorting results

What it is

Without `ORDER BY`, SQL returns rows in **no guaranteed order** — whatever is fastest for the database at that moment. `ORDER BY` is the only way to get a defined order.

```
SELECT column1, column2
FROM table_name
WHERE condition
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC];
```

`ASC` (ascending, smallest first) is the default; `DESC` is descending.

How it works

```
-- code/06_order_by.sql
SELECT first_name, salary FROM employees ORDER BY salary DESC;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
 Neha       | 142000.00
 Kiran      | 135000.00
 Lucas      | 120000.00
 Meera      |  98000.00
 Rahul      |  98000.00
 Omar       |  76000.00
 Priya      |  72000.00
 Arjun      |  68000.00
 Sara       |  68000.00
 Li         |  64000.00
 Ananya     |  61000.00
 Rohan      |  30000.00
(15 rows)
```

Meera and Rahul tie; their relative order is **not guaranteed** and may differ on another run. Add a **tie-breaker** column whenever order matters (especially for paging).

Several sort keys — the second key only matters within ties of the first, and each key has its own direction:

```
SELECT dept_id, first_name, salary FROM employees
ORDER BY dept_id, salary DESC, first_name;
```

Output:

```
 dept_id | first_name |  salary
---------+------------+-----------
       1 | Vikram     | 185000.00
       1 | Neha       | 142000.00
       1 | Meera      |  98000.00
       1 | Rahul      |  98000.00
       2 | Daniel     | 150000.00
       2 | Priya      |  72000.00
       2 | Arjun      |  68000.00
       2 | Sara       |  68000.00
       3 | Lucas      | 120000.00
       3 | Ananya     |  61000.00
       4 | Kiran      | 135000.00
       4 | Omar       |  76000.00
       5 | Li         |  64000.00
  (null) | Asha       | 250000.00
  (null) | Rohan      |  30000.00
(15 rows)
```

Where do NULLs go?

PostgreSQL treats NULL as **larger than any value**: NULLs come **last in ASC** and **first in DESC**. Override with `NULLS FIRST` / `NULLS LAST`:

```
SELECT name, location FROM departments ORDER BY location;
SELECT name, location FROM departments ORDER BY location NULLS FIRST;
SELECT name, location FROM departments ORDER BY location DESC NULLS LAST;
```

Output:

```
    name     | location
-------------+-----------
 Engineering | Bengaluru
 Sales       | Mumbai
 Marketing   | Mumbai
 Finance     | Pune
 Research    | Pune
 Support     | (null)
(6 rows)

    name     | location
-------------+-----------
 Support     | (null)
 Engineering | Bengaluru
 Sales       | Mumbai
 Marketing   | Mumbai
 Finance     | Pune
 Research    | Pune
(6 rows)

    name     | location
-------------+-----------
 Finance     | Pune
 Research    | Pune
 Sales       | Mumbai
 Marketing   | Mumbai
 Engineering | Bengaluru
 Support     | (null)
(6 rows)
```

Database	NULLs in ASC	`NULLS FIRST/LAST` supported?
PostgreSQL, Oracle	last	yes
MySQL, SQL Server, SQLite	first	SQLite yes (3.30+); MySQL/SQL Server no — use `ORDER BY col IS NULL, col` (MySQL) or a `CASE` expression

Sorting by alias, expression or position

```sql
SELECT name, unit_price * 1.18 AS gross FROM products ORDER BY gross DESC LIMIT 3;
SELECT first_name, last_name FROM employees ORDER BY length(last_name) DESC, last_name LIMIT 3;
SELECT first_name, hire_date FROM employees ORDER BY 2 LIMIT 3;
```

Output:

```
      name       |   gross
-----------------+-----------
 Laptop Pro 14   | 1532.8200
 Standing Desk   |  588.8200
 Ergonomic Chair |  412.9882
(3 rows)

 first_name | last_name
------------+-----------
 Daniel     | Brooks
 Omar       | Farouk
 Lucas      | Martin
(3 rows)

 first_name | hire_date
------------+------------
 Asha       | 2015-01-05
 Kiran      | 2016-12-01
 Daniel     | 2017-09-25
(3 rows)
```

`ORDER BY 2` means "the second column of the select list". It's quick in psql but fragile in code — reorder the select list and the sort silently changes.

Custom order with CASE

Status values should sort by workflow, not alphabetically:

```
SELECT order_id, status FROM orders
ORDER BY CASE status WHEN 'pending' THEN 1 WHEN 'shipped' THEN 2
                    WHEN 'delivered' THEN 3 ELSE 4 END, order_id;
```

Output (abridged):

```
 order_id |  status
----------+-----------
      113 | pending
      114 | pending
      107 | shipped
      110 | shipped
      112 | shipped
      101 | delivered
      ...
      109 | delivered
      105 | cancelled
      111 | cancelled
(14 rows)
```

(`CASE` is explained in chapter 3.)

Text order depends on collation

A **collation** is the rule set for comparing and sorting text in a language. The book's server uses the simple `C` collation, which sorts by byte value — so all capital letters come before all lower-case ones:

```
SELECT city FROM customers WHERE city IS NOT NULL ORDER BY city;
SELECT city FROM customers WHERE city IS NOT NULL ORDER BY lower(city), city;
```

Output:

```
   city
-----------
 Bengaluru
 Delhi
 Delhi
 London
 London
 Mumbai
 New York
 New York
 mumbai
(9 rows)

   city
-----------
 Bengaluru
 Delhi
 Delhi
 London
 London
 Mumbai
 mumbai
 New York
 New York
(9 rows)
```

A database created with a language collation such as `en_US.UTF-8` would sort `mumbai` next to `Mumbai` without help. You can also choose per query: `ORDER BY city COLLATE "en-x-icu"` (needs ICU support).

Common mistakes

- Relying on insertion order or "it always came back sorted" — without `ORDER BY` there is no order.
- Forgetting a unique tie-breaker, then paging (2.8) shows some rows twice and others never.
- Sorting numbers stored as text: `'10' < '9'` alphabetically. Store numbers in numeric columns.

Self-check

1. *Default direction?* → ASC.
2. *Where does PostgreSQL put NULLs with `ORDER BY x DESC`?* → First.
3. *Why add `emp_id` to `ORDER BY salary DESC`?* → As a unique tie-breaker so the order is deterministic.

TL;DR

Only `ORDER BY` guarantees order. Multiple keys, each `ASC`/`DESC`; control NULLs with `NULLS FIRST/LAST`; always add a unique tie-breaker when order matters; text sorting follows the collation.

2.8 LIMIT, OFFSET and FETCH: returning a page

What it is

`LIMIT n` returns at most `n` rows. `OFFSET m` skips the first `m` rows. Together they return "page *k*" of a result. The SQL-standard spelling is `OFFSET m ROWS FETCH FIRST n ROWS ONLY`. **Always pair them with `ORDER BY`** — "the first 3 rows" means nothing without an order.

How it works

```sql
-- code/07_limit_offset.sql
-- Top 3 earners
SELECT first_name, salary FROM employees ORDER BY salary DESC LIMIT 3;

-- Page 2 with 5 rows per page (skip 5)
SELECT emp_id, first_name FROM employees ORDER BY emp_id LIMIT 5 OFFSET 5;

-- Standard SQL spelling of the same thing
SELECT emp_id, first_name FROM employees ORDER BY emp_id
OFFSET 5 ROWS FETCH FIRST 5 ROWS ONLY;
```

Output:

```
 first_name |  salary
------------+-----------
 Asha       | 250000.00
 Vikram     | 185000.00
 Daniel     | 150000.00
(3 rows)

 emp_id | first_name
--------+------------
      6 | Daniel
      7 | Priya
      8 | Arjun
      9 | Sara
     10 | Lucas
(5 rows)

 emp_id | first_name
--------+------------
      6 | Daniel
      7 | Priya
      8 | Arjun
      9 | Sara
     10 | Lucas
(5 rows)
```

For page *p* (starting at 1) with page size *s*: `LIMIT s OFFSET (p - 1) * s`.

Ties: FETCH … WITH TIES

`LIMIT 1` on a tie picks one row arbitrarily. `FETCH FIRST n ROWS WITH TIES` (PostgreSQL 13+, requires `ORDER BY`) also returns every row that ties with the last one:

```sql
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary LIMIT 1;
SELECT first_name, salary FROM employees WHERE dept_id IN (1, 2)
ORDER BY salary FETCH FIRST 1 ROWS WITH TIES;
```

Output:

```
 first_name |  salary
------------+----------
 Arjun      | 68000.00
(1 row)

 first_name |  salary
------------+----------
 Arjun      | 68000.00
 Sara       | 68000.00
(2 rows)
```

"Lowest-paid person" has two correct answers; `WITH TIES` returns both. (Ranking functions in chapter 8 generalise this.)

Why OFFSET gets slow — keyset pagination

`OFFSET 100000` still makes the database produce and throw away 100,000 rows, so deep pages get slower and slower. Rows inserted or deleted between page loads also shift the pages, so users see duplicates or miss rows. **Keyset** (or "seek") pagination remembers the last key shown and asks for rows after it:

```sql
-- page after the one that ended at emp_id 10
SELECT emp_id, first_name FROM employees WHERE emp_id > 10 ORDER BY emp_id LIMIT 5;
```

Output:

```
 emp_id | first_name
--------+------------
     11 | Ananya
     12 | Kiran
     13 | Omar
     14 | Li
     15 | Rohan
(5 rows)
```

With an index on the key, every page costs the same no matter how deep. The trade-off: you can go "next/previous" but not jump to page 500. Details in chapter 12.

Dialect differences

Database	First 10 rows	Skip 20, take 10
PostgreSQL	`LIMIT 10` or `FETCH FIRST 10 ROWS ONLY`	`LIMIT 10 OFFSET 20` or `OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
MySQL	`LIMIT 10`	`LIMIT 10 OFFSET 20` or `LIMIT 20, 10` (offset first!)
SQLite	`LIMIT 10`	`LIMIT 10 OFFSET 20`
SQL Server	`SELECT TOP (10) …`	`ORDER BY … OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`
Oracle 12c+	`FETCH FIRST 10 ROWS ONLY`	`OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY`

Common mistakes

- `LIMIT` without `ORDER BY` — "random" rows that can change between runs.
- MySQL's `LIMIT 20, 10` read as "20 rows starting at 10" — it's the reverse.

- Paging on a non-unique sort key — rows at page boundaries repeat or vanish. Add a unique tie-breaker.

In the interview

> **30-second answer:** "`ORDER BY` plus `LIMIT n OFFSET m`, or the standard `OFFSET … FETCH`. For deep pages OFFSET is O(offset) and unstable under inserts, so I'd use keyset pagination — `WHERE (sort_key, id) > (:last_key, :last_id) ORDER BY sort_key, id LIMIT n` — backed by an index."

- *How do you get the top 3 including ties?* → `FETCH FIRST 3 ROWS WITH TIES`, or `DENSE_RANK()` (chapter 8).
- *SQL Server equivalent of LIMIT?* → `TOP (n)` or `OFFSET … FETCH`.

Self-check

1. *Rows returned by `LIMIT 5 OFFSET 12` over 15 rows?* → 3 (rows 13–15).
2. *Page 4, size 20?* → `LIMIT 20 OFFSET 60`.
3. *What does keyset pagination need?* → A stable, unique ordering key (and ideally an index on it).

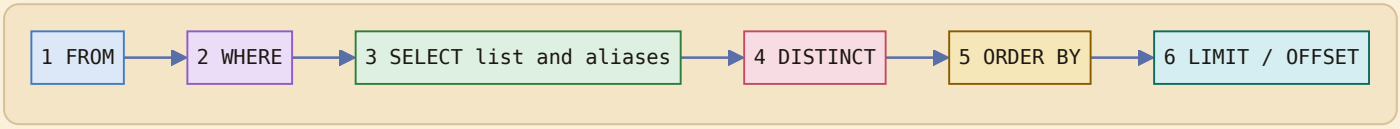
TL;DR

`ORDER BY … LIMIT n OFFSET m` (or `OFFSET … FETCH`). `WITH TIES` keeps ties. Deep `OFFSET` is slow and unstable; prefer keyset pagination.

2.9 How a SELECT is evaluated (logical processing order)

What it is

You *write* a query in the order `SELECT … FROM … WHERE … ORDER BY … LIMIT`, but the database *evaluates it logically* in a different order. Knowing that order explains many error messages.



(Chapter 4 inserts `GROUP BY` and `HAVING` between `WHERE` and `SELECT`, and window functions after them.)

Consequences:

- **`WHERE` cannot use a column alias** defined in the `SELECT` list, because `SELECT` hasn't happened yet.
- **`ORDER BY` can use aliases**, because it runs after `SELECT` — but in PostgreSQL only a bare alias: `ORDER BY gross` works, `ORDER BY gross + 0` fails with *column "gross" does not exist* (inside an expression, names refer to input columns).
- `LIMIT` is applied last, to the already sorted result.

```sql
-- code/01_select_columns.sql (end)
SELECT name, unit_price * 1.18 AS gross FROM products WHERE gross > 500;
SELECT name, unit_price * 1.18 AS gross FROM products WHERE unit_price * 1.18 > 500 ORDER BY gross
DESC;
```

Output:

```
ERROR:  column "gross" does not exist
LINE 1: ...e, unit_price * 1.18 AS gross FROM products WHERE gross > 50...
                                                             ^
     name      |   gross
---------------+-----------
 Laptop Pro 14 | 1532.8200
 Standing Desk |  588.8200
(2 rows)
```

This is the *logical* order that defines the meaning; the optimizer may physically do things differently (e.g. use an index to read rows already sorted) as long as the result is the same.

Self-check

1. *Why does* `WHERE gross > 500` *fail?* → `WHERE` is evaluated before the `SELECT` list defines `gross`.
2. *Is* `ORDER BY gross` *allowed?* → Yes; `ORDER BY` comes after `SELECT`.

TL;DR

Logical order: FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT. Aliases exist from SELECT onward.

Putting it together

A query using every clause from this chapter — "the three most recent non-cancelled 2025 orders placed through a sales rep":

```sql
-- code/09_putting_it_together.sql
SELECT order_id, customer_id, sales_rep_id, placed_at, status
FROM orders
WHERE status <> 'cancelled'
  AND sales_rep_id IS NOT NULL
  AND placed_at >= '2025-01-01' AND placed_at < '2026-01-01'
ORDER BY placed_at DESC, order_id DESC
LIMIT 3;
```

Output:

```
 order_id | customer_id | sales_rep_id |      placed_at      | status
----------+-------------+--------------+---------------------+---------
      113 |           1 |            7 | 2025-06-02 10:30:00 | pending
      112 |           5 |            9 | 2025-05-20 21:05:00 | shipped
      110 |           7 |            8 | 2025-04-25 11:10:00 | shipped
(3 rows)
```

Chapter 2 TL;DR

- `SELECT cols FROM t` — name your columns; expressions and `AS` aliases are free; table aliases shorten references.
- `WHERE` keeps TRUE rows only. Precedence `NOT` > `AND` > `OR`: parenthesise.
- Integer division truncates in PostgreSQL (`7/2 = 3`); `||` concatenates; compound `+=` is not standard SQL.

- `BETWEEN` is inclusive (use half-open ranges for timestamps); `IN` = ORs; `LIKE` (`%`, `_`) is case-sensitive, `ILIKE` isn't; `~` for regex.
- NULL means unknown → three-valued logic. Use `IS [NOT] NULL`, `IS DISTINCT FROM`; beware `NOT IN` with NULLs.
- `DISTINCT` removes exact duplicate rows (NULLs grouped); `DISTINCT ON` keeps first row per group.
- Only `ORDER BY` gives order; add tie-breakers; `NULLS FIRST/LAST`; collation affects text.
- `LIMIT/OFFSET` or `OFFSET … FETCH`; `WITH TIES`; keyset pagination for deep pages.
- Logical order FROM → WHERE → SELECT → DISTINCT → ORDER BY → LIMIT.

Next: Chapter 3 — Functions and expressions.